Katedra Telekomunikacji
Multimedialnej i Mikroelektroniki

Laboratorium

Programowalne Układy Cyfrowe

Ćwiczenie 8 – Liczby zespolone

Materiały dydaktyczne przeznaczone są WYŁĄCZNIE dla studentów wydziału Elektronika i Telekomunikacja Politechniki Poznańskiej

Cele:

- zrealizowanie samodzielnej implementacji modułu,
- zapoznanie z funkcjami wczytywania/zapisu do plików,
- moduł mnożenia zespolonego.

Przebieg ćwiczenia:

1. Uruchomić program Active-HDL, stworzyć nowy projekt.
2. Wprowadzić następujące pliki z wektorami testowymi, zawierające:
   - w kolejnych wierszach - kolejne wektory testowe,
   - w kolejnych kolumnach:: A_{re} A_{im} B_{re} B_{im} (input.txt) C_{re} C_{im} (output.txt), takie, że $(A_{re} + j\,A_{im})^* (B_{re} + j\,B_{im}) = C_{re} + j\,C_{im}$

Plik input.txt:

```
        12      3       1       0
        0       1       0       1
.....
```

Plik output.txt:

```
        12      3
        -1      0
.....
```

Pliki powinny zawierać co najmniej 16 przykładów.

3. Wpisać program poniżej:

Plik testbench.v

```
module mytestbenchmodule();
reg CLK;
initial CLK  <= 0;
always #50  CLK <= ~CLK;

reg RST;
initial
begin
        RST <= 0;
        RST <= #50 1;
        RST <= #500 0;
end

integer fi,fo;
integer count;
reg error;
integer  f1, f2, f3, f4;


reg istb;
reg [31:0] vec_Are, vec_Aim, vec_Bre, vec_Bim, vec_Cre, vec_Cim;
```

```
wire ostb;

wire [31:0] Cre, Cim;

initial
begin
        fi = $fopen("input.txt","r");
        fo = $fopen("output.txt","r");
        istb = 0;
        error = 0;
end


always @(posedge CLK or posedge RST)
if (!RST) begin
        count = $fscanf(fi, "%d %d %d %d", f1,f2,f3,f4); // vec_Are, vec_Aim, vec_Bre, vec_Bim
        if (count==4)
        begin
                istb <= 1;
                vec_Are <= f1;
                vec_Aim  <= f2;
                vec_Bre <= f3;
                vec_Bim <= f4;
        end else istb<=0;
end

always @(posedge CLK or posedge RST)
if (!RST) begin
        if (ostb)
        count = $fscanf(fo, "%d %d", vec_Cre, vec_Cim);
        if (count==2)
        begin
                error = 0;
                if ((vec_Cre!=Cre) || (vec_Cim!=Cim))
                begin
                        $display("error %d!=%d %d!=%d", vec_Cre,Cre,vec_Cim,Cim);
                        error = 1;
                end
        end
end

complex_mul      cmul
(
.CLK(CLK),
.RST(RST),
.I_STB(istb),
.I_ARE(vec_Are),
.I_AIM(vec_Aim),
.I_BRE(vec_Bre),
.I_BIM(vec_Bim),

.O_STB(ostb),
.O_CRE(Cre),
.O_CIM(Cim)
);

endmodule
```

Plik complex_mul.v::

```
    module complex_mul
    (
    input  wire        CLK,
    input  wire        RST,
    input  wire         I_STB,
    input  wire  [31:0] I_ARE,
    input  wire  [31:0] I_AIM,
    input  wire  [31:0] I_BRE,
    input  wire  [31:0] I_BIM,

    output reg         O_STB,
    output reg  [31:0] O_CRE,
    output reg  [31:0] O_CIM
    );


    always @(posedge CLK or posedge RST) O_STB <= (RST) ? 0 : I_STB;
    always @(posedge CLK or posedge RST) O_CRE <= (RST) ? 0 : I_ARE*I_BRE - I_AIM*I_BIM;
    always @(posedge CLK or posedge RST) O_CIM <= (RST) ? 0 : I_AIM*I_BRE + I_ARE*I_BIM;

    endmodule
```

4. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu testbench.
5. Czy układ działa poprawnie? Zweryfikować, czy zadanie błędnych wektorów testowych (pliki input.txt i output.txt) wygeneruje stosowny komunikat na konsoli i ustawi flagę "error" na 1.
6. Napisać samodzielnie moduł complex_mul taki, aby:
   - składał się z układów mnożących i sumujących poznanych na wcześniejszych zajęciach,
   - uwzględniał znaki liczb,
   - posiadał linie zajętości modułu "output wire I_BSY" (ustawiana na 1 gdy moduł jest zajęty i nie może przyjąć danych) - można skorzystać z poznanego modułu FIFO,
   - linie "input wire O_ACK" która potwierdza modułowi odbiór danych na wyjściu.
7. Zaprezentować zrealizowane zadanie prowadzącemu.